EcoCompliance API — Testes Automatizados

Projeto Maven separado. A API sobe no Docker e o Cucumber chama os endpoints com Rest Assured.

Integrante	RM
Évelyn Santos Rodrigues	565919
Fagner Nunes Lopes	564220
Lucas Fernando da Silva	563626
Raphael Salomão Chamma	564765
Ricardo Negrão da Veiga	564106

1. Como a suíte conversa com a API

O projeto testes-automatizados fica fora do código de produção. O docker compose sobe o Oracle e a API. O TestRunner, com a tag @regressivo, dispara os cenários. Os passos em Java usam o Rest Assured para enviar GET, POST, PUT e DELETE. A resposta real é conferida no status, em um campo do JSON e no JSON Schema Draft-07 gravado em src/test/resources/schemas. O pacote model guarda o cadastro da empresa, services concentra a chamada HTTP e a validação do contrato, steps traduz o Gherkin e hook registra o início e o fim de cada cenário.

As dependências seguem o capítulo da fase: cucumber-java e cucumber-junit 7.18.1, rest-assured 5.5.0, gson 2.11.0, lombok 1.18.34 e json-schema-validator 1.5.1. O Surefire executa só a classe TestRunner, com mvn clean test.

2. Como executar

```
docker compose up --build -d
cd testes-automatizados
./scripts/executar.sh
mvn clean test
```

O script espera GET /up e então roda o Maven. Outra base usa a variável API_BASE_URL. O relatório HTML sai em testes-automatizados/target/cucumber-reports.html. O workflow .github/workflows/testes-automatizados.yml sobe o Compose, espera a API e executa a mesma suíte.

3. Cenários de governança

Cinco cenários, em português, com caminho feliz e caminho de falha. O cadastro válido e o login confirmam o fluxo que a governança precisa. A validação, a autorização e o recurso inexistente confirmam a recusa.

```
# language: pt
@regressivo
Funcionalidade: Governança ambiental da empresa
  A API registra empresas e controla quem pode alterar o cadastro.
  Os cenários cobrem o caminho feliz e as falhas de validação, acesso e recurso inexistente.

  Cenário: Cadastrar empresa com dados válidos
    Dado um administrador autenticado
    Quando ele cadastra uma empresa válida
    Então a API responde 201
    E o campo "nome" está preenchido
    E o contrato "empresa.json" é respeitado
    E o cabeçalho Location aponta para o recurso

  Cenário: Recusar empresa sem nome
    Dado um administrador autenticado
    Quando ele envia um cadastro de empresa sem nome
    Então a API responde 400
    E o campo "message" vale "Erro de validação nos campos"
    E o contrato "erro.json" é respeitado
  Cenário: Usuário comum não cadastra empresa
```

```
  Dado um usuário comum autenticado
  Quando ele cadastra uma empresa válida
  Então a API responde 403
  E o corpo vem vazio

Cenário: Consultar empresa inexistente
  Dado um administrador autenticado
  Quando ele busca a empresa de id 999999
  Então a API responde 404
  E o contrato "erro.json" é respeitado

Cenário: Entrar com credenciais válidas
  Quando o administrador entra com e-mail e senha
  Então a API responde 200
  E o campo "tipo" vale "Bearer"
  E o campo "token" está preenchido
  E o contrato "auth.json" é respeitado
```

3.1 Cadastrar empresa com dados válidos

Um administrador envia nome, CNPJ, setor e e-mail. A API responde 201, o corpo traz o nome, o schema empresa.json fecha o contrato e o cabeçalho Location aponta para /empresas/{id}.

3.2 Recusar empresa sem nome

O nome vai em branco. A API responde 400, a mensagem é Erro de validação nos campos e o corpo segue erro.json, com timestamp, status, error, message, path e fieldErrors.

3.3 Usuário comum não cadastra empresa

O usuário user@ecocompliance.com tenta o mesmo cadastro. A API responde 403 com corpo vazio: o filtro de segurança barra a escrita antes do controller. POST, PUT e DELETE continuam restritos ao papel ADMIN.

3.4 Consultar empresa inexistente

A busca pelo id 999999 responde 404 e o corpo segue erro.json. A consulta não inventa um cadastro.

3.5 Entrar com credenciais válidas

O login público responde 200 com token, tipo Bearer, e-mail, nome e role. O schema auth.json fecha esse contrato.

4. Contrato de todas as APIs

O arquivo contrato.feature repete o mesmo critério em cada recurso: status, um campo do JSON e o schema. Listas paginadas passam por pagina.json e pelo schema do item. Listas simples conferem cada elemento. DELETE responde 204 sem corpo. Os dados de cada cenário usam CNPJ e número de licença novos, para a suíte poder rodar de novo.

Cenário	O que a requisição real confere
Saúde da API	GET /up, 200, status up, up.json
Ambiente publicado	GET /ambiente, 200, aplicacao, ambiente.json
Recusar login sem senha	POST /auth/login, 400, erro.json
Cadastrar usuário	POST /auth/register, 201, token Bearer, auth.json
Empresas	POST, GET, página, PUT e DELETE 204
Licenças	CRUD, GET /licencas e GET /licencas/vencendo
Auditorias	CRUD, página e GET /auditorias/risco/ALTO
Emissões	CRUD, página e GET /emissoes/empresa/{id}
Compensações	CRUD e página

Cenário	O que a requisição real confere
Agendamentos	CRUD, página, filtro por empresa e por status
Relatório	GET /relatorios/empresa/{id}, maior risco ALTO
Indicadores	GET /indicadores-sustentabilidade

5. Evidência da execução

A suíte rodou em 1º de novembro de 2026, às 21:02, no fuso -03, contra a API em http://localhost:8080, publicada pelo docker compose com o Oracle saudável. O Maven registrou 17 cenários, 0 falhas, 0 erros e BUILD SUCCESS, em 4,4 s.

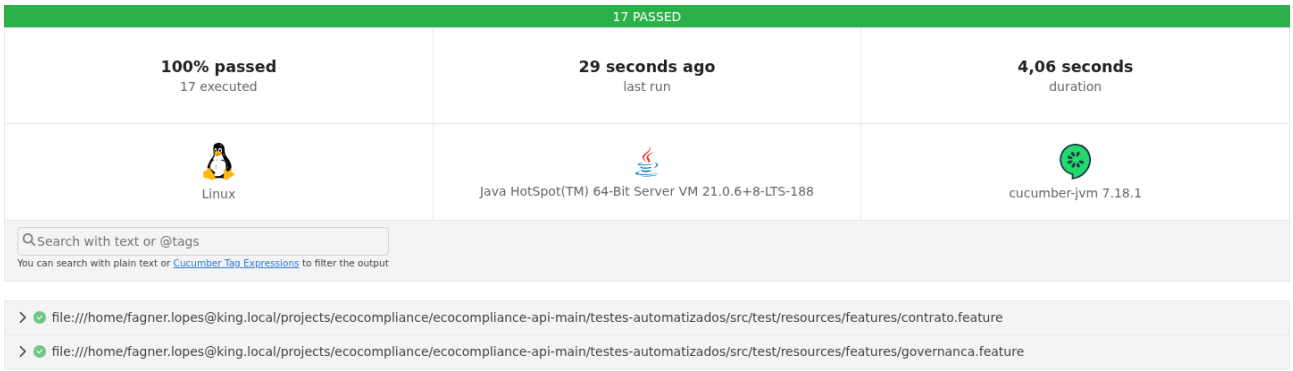


Figura 1. Relatório target/cucumber-reports.html. Cucumber 7.18.1, 17 cenários executados, 100% aprovados, nas features contrato.feature e governanca.feature.

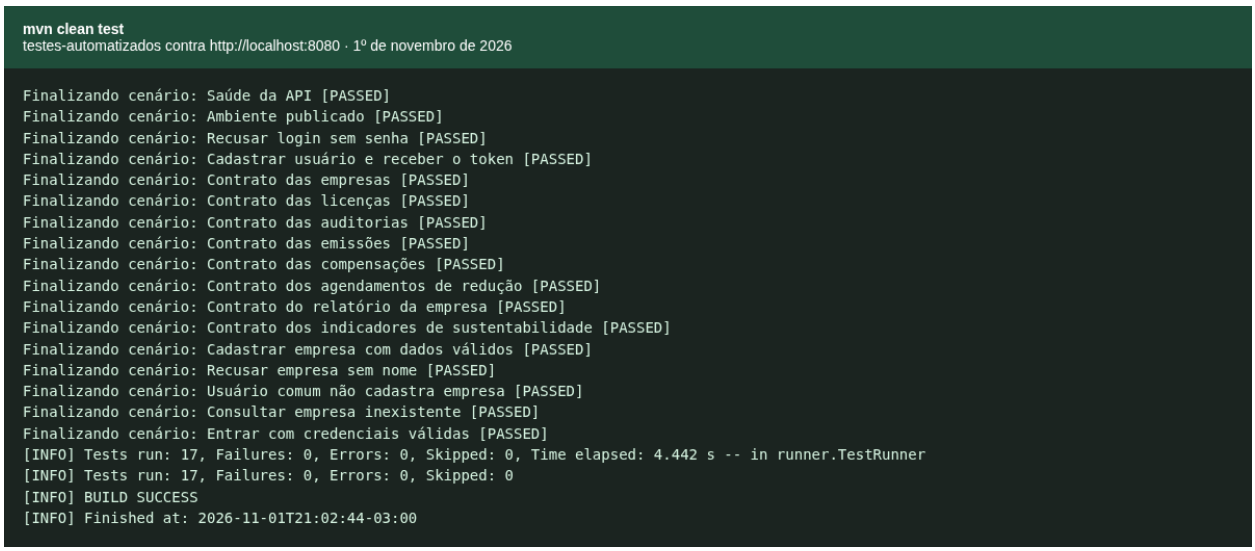


Figura 2. Fechamento de mvn clean test no TestRunner. Cada cenário terminou PASSED e a construção encerrou com BUILD SUCCESS.